
BudgetFlow: Value-Aware Budget Governance for Agent Tasks

Fengde Ding¹

Abstract

Multi-step AI agent tasks consume variable amounts of model budget, and the tasks in a batch often have unequal value. Existing model-routing work primarily decides which model should answer one request or how one task should spend its own budget. We study *Batch-Level Task Budget Allocation*: a task batch shares one hard budget, and the system must allocate model opportunities across tasks before the budget is exhausted.

We introduce BudgetFlow, a value-aware budget governance framework for task batches. Each task exposes four interfaces: Task Value, Estimated Token Demand, Model Fit, and a Verifier. BudgetFlow uses the first three signals to allocate budget and model tiers, while the Verifier settles outcomes. We evaluate BudgetFlow on SWE-bench-style verified bug-fixing tasks under fixed task sets and shared hard budgets. The paper reports three primary metrics: Resolved Rate, Total Resolved Value (TRV), and TRV per Dollar.

Across one 4-policy run and three 5-policy runs over 30-task batches, BudgetFlow wins the latest audited run on all three primary metrics, wins a 4-policy run on TRV and TRV per Dollar, and closely approaches the strong-model-only boundary in two runs where the strong model is unusually cost-effective. Sensitivity analyses over Task Value, budget cap, and KV-cache cost discount show how the cost-value frontier changes under different operating conditions. These results support BudgetFlow as a concrete formulation of value-aware task-batch budgeting, with SWE-bench as a verified coding instantiation.

1. Introduction

AI agents increasingly execute multi-step tasks: they inspect context, call tools, edit artifacts, run checks, and decide

¹University of International Relations. Correspondence to: Fengde Ding <fengde.ding@uir.edu.cn>.

Preprint. July 7, 2026.

whether another model call is worth the cost. These tasks consume budget unevenly. One task may finish after a short reasoning loop; another may require repeated search, repair, and verification. In a batch, this variation means that a fixed model budget can be exhausted before every task receives the same opportunity to succeed.

A central issue is that tasks rarely have equal value. A support ticket with an imminent service-level agreement, a security patch, a revenue-critical spreadsheet fix, and a routine rewrite should receive different budget treatment when they are submitted in the same batch. Once model calls become a scarce shared resource, the system needs an allocation policy in addition to a model router.

This paper studies *Batch-Level Task Budget Allocation*. The general setting has many tasks, one shared hard budget, model tiers with different costs and capabilities, and an outcome rule that decides whether a task is resolved. The objective is to allocate scarce model opportunities so the batch produces more verified task value under the same resource constraint.

The operational decision is simple: which task should receive the next scarce model opportunity, under the same shared budget?

BudgetFlow is a budget governance framework for this setting. It treats each task as exposing four interfaces:

- **Task Value:** the pre-registered value gained if the task is resolved;
- **Estimated Token Demand:** the pre-run estimate of token/runway demand;
- **Model Fit:** evidence about how well each model tier fits the task;
- **Verifier:** the outcome rule that maps a completed attempt to resolved or unresolved.

The first three interfaces drive allocation decisions. The Verifier settles outcomes and defines the objective. This separation is the core interface. The policy can change how it routes, caps, escalates, or stops tasks, while the outcome contract remains explicit.

We instantiate this formulation on SWE-bench-style verified coding tasks. SWE-bench is useful here because it gives a strict task-batch testbed: each task is a real bug-fixing issue, each attempt has measurable model spend, and each outcome is settled by a test-based harness. The benchmark is the evidence domain, while the allocation interface is the research object.

The paper uses three primary metrics. **Resolved Rate** anchors the evaluation to standard verified success. **Total Resolved Value** (TRV) sums pre-registered Task Value over resolved tasks. **TRV per Dollar** measures value efficiency under the shared hard budget.

Our results are boundary-aware. BudgetFlow wins the latest audited 5-policy, 30-task run on Resolved Rate, TRV, and TRV per Dollar. It wins a 4-policy run on value and value efficiency while resolving one fewer task than the strong-model-only baseline. The strong-model-only policy leads in conditions where the strong model itself forms a strong cost-value boundary. These outcomes are reported together because the scientific object is the cost-value frontier across operating conditions.

This paper makes three contributions:

1. We formulate Batch-Level Task Budget Allocation with pre-registered Task Value, Estimated Token Demand, Model Fit, a shared hard budget, and a Verifier-settled objective.
2. We present BudgetFlow, a task-batch budget governance framework that allocates model opportunities using value, estimated demand, model fit, and remaining budget.
3. We evaluate BudgetFlow against cheap-model-only, strong-model-only, learned task-router, and budget-only baselines, reporting Resolved Rate, TRV, TRV per Dollar, and sensitivity curves that expose operating conditions.

2. Related Work

Per-query routing and cascading. RouteLLM learns per-query routers from preference data (Ong et al., 2025). Cascade Routing studies model routing and cascading for individual queries (Dekoninck et al., 2025). RouteNLP targets enterprise query routing under quality constraints (Guo et al., 2026). UCCI calibrates uncertainty for cost-optimal cascade routing (Kotte, 2026). These works make model selection a first-class cost-quality problem. The unit is typically one prompt or one request. BudgetFlow operates at the task-batch layer: many multi-step tasks share one hard budget, and the policy allocates scarce model opportunities across tasks with different values.

Within-task budgeted agents. INTENT studies budget-constrained tool use inside a single agent task (Liu et al., 2026). This is a complementary layer to Batch-Level Task Budget Allocation. A within-task policy decides whether one task should spend another tool or model call. BudgetFlow decides how one budget should be shared across many tasks before the batch budget is exhausted.

Coding-agent evaluation and cost-aware runtimes. SWE-bench provides the verified coding-task substrate used by many agent evaluations (Jimenez et al., 2024). OpenSquilla is a token-efficient agent runtime with model routing, diagnostics, replay, and cost-per-success framing (OpenSquilla contributors, 2026). Claw-SWE-Bench studies harnesses, adapters, prompts, patch extraction, leakage cleanup, and cost reporting for SWE-bench-style coding-agent evaluation (Zheng et al., 2026). These systems sharpen the evaluation requirements for agent tasks: cost, harness behavior, and outcome verification must be explicit.

Serving substrates. vLLM improves LLM serving through PagedAttention and KV-cache management (Kwon et al., 2023); SGLang accelerates structured language-model programs with runtime optimizations such as KV-cache reuse (Zheng et al., 2024); Dynamo exposes agentic inference concerns including cache locality and scheduling (NVIDIA, 2026). These systems provide execution substrates. BudgetFlow provides task-level value and budget signals that can inform priority, continuation, model opportunity, and cache-sticky scheduling in future serving-aware systems.

3. Problem Formulation

Let a task batch contain tasks $\mathcal{T} = \{1, \dots, n\}$ and let B denote the shared hard budget. Each task i has a pre-registered value $v_i > 0$, an estimated token demand $d_i > 0$, and model-fit evidence $m_{i,k}$ for each model tier k . A policy π chooses a sequence of actions that spend budget on model calls and task attempts. Let $c_i(\pi)$ be the spend on task i , and let

$$\sum_{i=1}^n c_i(\pi) \leq B$$

be the shared hard-budget constraint.

The budget is shared at the batch level. A policy can spend more on one task only by leaving less budget for later tasks in the same batch. This coupling is what separates the problem from independent per-task routing: a locally reasonable strong-model call can be globally poor if it consumes budget needed by a higher-value task.

Each task has a Verifier V_i that maps the final task evidence

to a binary outcome:

$$r_i(\pi) = V_i(\pi, i) \in \{0, 1\}.$$

The primary value objective is Total Resolved Value:

$$\text{TRV}(\pi) = \sum_{i=1}^n v_i r_i(\pi).$$

We report three headline metrics:

$$\text{ResolvedRate}(\pi) = \frac{1}{n} \sum_{i=1}^n r_i(\pi),$$

$$\text{TRVperDollar}(\pi) = \frac{\text{TRV}(\pi)}{\sum_i c_i(\pi)}.$$

The Verifier is an outcome settlement rule. The policy uses Task Value, Estimated Token Demand, and Model Fit for allocation; the Verifier settles the final outcome after execution. In our SWE-bench instantiation, V_i is the test-based harness. This gives a strict external success signal while preserving the task-batch allocation structure.

The four interfaces have different portability requirements. Task Value must be fixed before execution. Estimated Token Demand is a pre-run planning estimate; final spend is measured after execution. Model Fit can come from catalog priors, historical outcomes, or calibration runs. The Verifier must be a trusted outcome rule for the domain. Their concrete calibration changes by domain, but the allocation problem consumes the same interface.

3.1. Core Terms and Metric Reading

Table 2 gives the compact reading guide used throughout the paper. The metrics are intentionally simple: success is verifier-settled, value is pre-registered, and efficiency is value divided by spend. This makes the frontier interpretable through standard verified outcomes and a pre-registered value ledger.

4. BudgetFlow

BudgetFlow maintains a shared budget ledger across the task batch. For each task, it estimates whether the next model opportunity should use the cheap model, the strong model, or defer additional spend. The ledger is hard: the remaining batch budget gates each planned action.

The policy uses three allocation inputs:

- v_i , the Task Value;
- d_i , the Estimated Token Demand;
- $m_{i,k}$, the Model Fit for tier k .

These inputs enter distinct decisions. Task Value prioritizes tasks that create more value if verified. Estimated Token Demand becomes runway and cap planning: the policy estimates how much budget a task may need before it can produce a useful artifact. Model Fit estimates whether the strong model is likely to improve the outcome relative to the cheap model. Remaining budget controls affordability and stop/defer decisions.

Let h denote budget pressure, with higher h indicating less remaining headroom. Let k_c be the cheap model and k_s be the strong model. A simplified task-start score for giving task i a strong-model opportunity is

$$S_i = \frac{v_i \cdot \max(m_{i,k_s} - m_{i,k_c}, 0) \cdot g(d_i)}{\max(\hat{c}_{i,k_s} - \hat{c}_{i,k_c}, \epsilon)} \cdot q(h),$$

where $\hat{c}_{i,k}$ is the estimated cost for tier k , $g(d_i)$ scales the opportunity by estimated runway, and $q(h)$ reduces strong-model access as the shared budget tightens. The implementation uses this score with affordability checks, learned task-router priors, and hard-budget gates. The formula captures the core allocation principle: strong-model budget is most valuable when a task has high value, sufficient expected model-fit gain, enough estimated runway, and enough remaining budget.

BudgetFlow separates allocation from outcome settlement. It uses Task Value, Estimated Token Demand, Model Fit, and the budget ledger to choose model opportunities. The Verifier evaluates the resulting patch or artifact after the task attempt. This design keeps the value objective auditable: the policy can explain why a task received budget, and the verifier independently determines whether that spend produced resolved value.

Decision surface. At task start, BudgetFlow assigns a route and an initial cap from value, estimated runway, model fit, and remaining budget. During execution, it can escalate high-value tasks when the strong model has expected upside and sufficient budget remains. It can stop or defer when additional spend has low expected value or would threaten the shared budget. These decisions are intentionally task-level in the current paper; finer stage or segment routing is left for future work.

Algorithm 1 is the implementation-facing view of the framework. The policy reads the three allocation interfaces before and during execution, while the Verifier is applied after the task attempt. This keeps routing, cap planning, and escalation auditable under the same value ledger used for evaluation.

5. Experimental Setup

Task set. We evaluate on fixed 30-task SWE-bench-style bug-fixing batches. Each task requires an agent to inspect

Table 1. Domain interfaces for Batch-Level Task Budget Allocation. The paper evaluates the SWE-bench instantiation.

Interface	SWE-bench instantiation	Other task-batch instantiations
Task Value	pre-registered criticality value	business priority, user tier, task owner value
Estimated Token Demand	pre-run token/runway estimate from task metadata	expected context size, tool steps, artifact size
Model Fit	tier evidence from prior runs and catalog priors	domain-specific success prior by model tier
Verifier	test-based harness	human review, rule check, delayed business outcome

Table 2. Core terms and metrics. The examples show how to read the reported numbers.

Term	Meaning	Example reading
Batch-Level Task Budget Allocation	A batch of tasks shares one hard budget, and a policy allocates model opportunities across the batch.	A 30-task batch spends at most \$9.95 total.
Task Value	Pre-registered value gained if the task passes the verifier.	A critical task can carry more value than a routine task.
Estimated Token Demand	Pre-run estimate of token/runway need, used for cap planning.	A broad repository fix may need a larger cap than a local edit.
Model Fit	Evidence about which model tier is likely to help this task.	Strong-model access is more useful when expected uplift is high.
Verifier	Domain outcome rule that settles whether the final artifact is resolved.	SWE-bench uses a test-based harness.
Resolved Rate	Fraction of tasks that pass the verifier.	16/30 means 53.3% resolved.
Total Resolved Value (TRV)	Sum of Task Value over verifier-passed tasks.	Values 2.0, 1.5, and 1.0 sum to TRV 4.5.
TRV per Dollar	Total verified value divided by total spend.	TRV 18.0 at \$9.95 gives 1.81 TRV/\$.

Algorithm 1 BudgetFlow batch policy sketch

- 1: Initialize remaining budget $R \leftarrow B$.
- 2: **for** task i in the batch **do**
- 3: Read $(v_i, d_i, m_{i,k_c}, m_{i,k_s})$ and current R .
- 4: Set an initial task cap from Task Value, Estimated Token Demand, and R .
- 5: Choose cheap or strong model from Model Fit, value upside, and affordability.
- 6: Execute the task while updating spend and remaining budget.
- 7: Escalate, continue, stop, or defer using value upside and budget pressure.
- 8: Settle the final artifact with V_i and record r_i .
- 9: **end for**
- 10: Report Resolved Rate, TRV, TRV per Dollar, and total spend.

a repository, edit code, and produce a patch. Outcomes are scored by the local harness and cross-checked with exported official predictions where available. Task values are pre-registered before execution.

Policies. Table 3 compares the five policies by the inputs they use. All policies run under the same shared hard budget and the same verifier. The key contrast is whether Task Value enters allocation of scarce model opportunities.

The 4-policy run uses the four policies available in that experimental line. All mainline policies use the same task order, budget protocol, model-tier catalog, and scoring path inside a run.

Metrics. The main tables report Resolved Rate, TRV, and TRV per Dollar, as defined in Table 2. Resolved Rate is written as both count and percentage. Spend is included as budget context because TRV per Dollar depends on it.

Sensitivity analyses. We report three sensitivity families on the latest audited 5-policy run. Task Value sensitivity re-scores fixed outcomes under alternate value profiles. Budget sensitivity replays the observed task order under different shared budget caps. KV Cache Cost-Discount sensitivity re-costs repeated input tokens under explicit cache-discount assumptions. These analyses use fixed observed outcomes and expose how the cost-value frontier changes under alternate accounting assumptions.

6. Results

6.1. Main Paid Evidence

Table 4 summarizes four paid task-batch runs. Red entries mark the positive BudgetFlow signals that should anchor the reader’s attention. The latest audited 5-policy run is the strongest positive signal: BudgetFlow resolves 16/30 tasks,

Table 3. Policy inputs and shared outcome contract. Runway/cap planning is the decision-facing use of Estimated Token Demand.

Policy	Same budget	Task Value	Runway/cap planning	Model choice	Same verifier
cheap-model-only	yes			fixed cheap model	yes
strong-model-only	yes			fixed strong model	yes
learned task-router	yes			value-blind learned choice	yes
budget-only	yes		yes	budget-pressure choice	yes
BudgetFlow	yes	yes	yes	value-aware choice	yes

reaches TRV 18.00, and achieves 1.81 TRV per Dollar. The next-best learned task-router resolves 15/30 tasks, reaches TRV 17.00, and achieves 1.71 TRV per Dollar.

The runs show three operating conditions. First, value-aware allocation can protect high-value tasks and improve TRV and TRV per Dollar even when it resolves one fewer task than strong-model-only. Second, a strong model can be the best boundary when it is both capable and cost-effective under the cap. Third, with the latest audited policy, BudgetFlow improves all three primary metrics against the five-policy table.

These outcomes should be read together. The 4-policy run is the clearest value-aware allocation case: raw resolved count alone favors the strong-model-only baseline, but the resolved tasks under BudgetFlow carry more value and cost less. The two 5-policy boundary runs show pressure from strong controls. The latest audited 5-policy run shows the desired frontier movement: BudgetFlow leads on standard resolution, value, and value per dollar under the same cap.

6.2. Full Latest 5-Policy Table

Table 5 gives the full latest audited run. BudgetFlow leads all three primary metrics. This table is the main point comparison; the next section adds sensitivity curves to show how the ordering changes under alternate operating conditions.

6.3. Task-Level Model Advantage

The latest run contains real model-tier heterogeneity. Among tasks where both cheap and strong model rows consumed model budget, six tasks are cheap-model-favorable passes and five are strong-model-favorable passes; seven comparable tasks fail under both. This explains why all-simple policies are useful controls: the strong model often helps, but the task batch also contains tasks where cheap-model execution is the better use of budget.

This task-level heterogeneity motivates selective strong-model use. When strong-model upside is concentrated on a subset of tasks, the allocation policy should preserve strong-model opportunities for those tasks while spending less on tasks where the cheap model is sufficient or both tiers are unlikely to pass.

7. Sensitivity Analysis

7.1. Task Value Sensitivity

Table 6 re-scores the latest audited run under alternate Task Value profiles. BudgetFlow remains ahead of the best control under equal value, criticality value, compressed criticality, and expanded criticality. A value permutation diagnostic over 64 permutations gives BudgetFlow a positive margin in all samples, with median margin +1.00 TRV.

7.2. Budget Sensitivity

Figure 2 shows budget sensitivity over the latest audited run. The curve fixes observed outcomes and replays budget accounting at different caps. It therefore describes the observed cost-value frontier for this completed run. The strong-model-only policy is strongest at some low and mid caps, while BudgetFlow leads at the full cap in the paid run.

7.3. KV Cache Cost-Discount Sensitivity

Table 7 re-costs repeated input tokens under KV-cache discounts. BudgetFlow leads or is close to the best control through KV90. At KV98 and KV99 the learned task-router has the highest TRV per Dollar, while BudgetFlow keeps a higher TRV than the learned task-router in the fixed-outcome table. This result is useful for serving-aware extensions: cache pricing can change the efficiency frontier while verified outcomes stay fixed.

8. Discussion and Future Work

Formulation-level generalization. The SWE-bench experiments instantiate the formulation in a verified coding domain. The formulation applies to task domains where tasks expose Task Value, Estimated Token Demand, Model Fit, and a Verifier. Coding tasks use tests as the Verifier. Support tasks can use policy checks and service outcomes. Spreadsheet tasks can use cell-level checks or rule engines. Research and document-production tasks can use human review or domain checks. The calibration of each interface is domain-specific; the allocation objective and budget ledger are shared.

Table 4. Main paid evidence. Each row reports the leader for one 30-task batch. Resolved Rate, TRV, and TRV per Dollar are the three primary metrics.

Run	Leading policy	Resolved Rate	Spend	TRV	TRV per Dollar
4-policy value-aware allocation case	BudgetFlow	14/30 (46.7%)	\$7.52	18.50	2.46
	strong-model-only	15/30 (50.0%)	\$9.46	18.00	1.90
5-policy strong-model boundary case	strong-model-only	17/30 (56.7%)	\$9.95	20.00	2.01
	BudgetFlow	15/30 (50.0%)	\$9.95	17.50	1.76
5-policy learned-router stress case	strong-model-only	18/30 (60.0%)	\$9.88	19.00	1.92
	BudgetFlow	17/30 (56.7%)	\$9.95	18.50	1.86
Latest audited 5-policy positive case	BudgetFlow	16/30 (53.3%)	\$9.95	18.00	1.81
	learned task-router	15/30 (50.0%)	\$9.95	17.00	1.71
	strong-model-only	15/30 (50.0%)	\$9.95	16.50	1.66

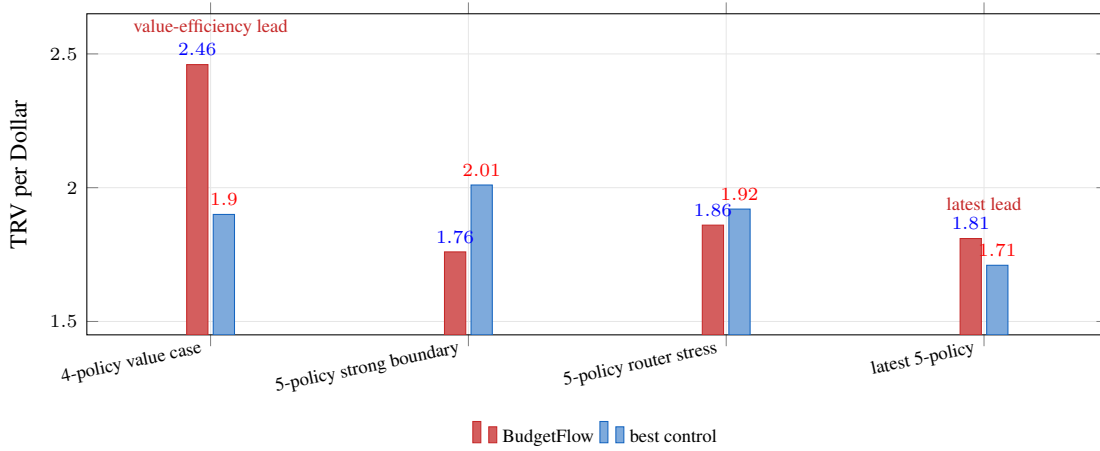


Figure 1. Multi-run main evidence. Each pair compares BudgetFlow with the best available control in that run on TRV per Dollar. The figure separates the repeated-run evidence from the sensitivity analysis in Section 7.

Table 5. Latest audited 5-policy run over 30 tasks.

Policy	Resolved Rate	TRV	TRV/\$
BudgetFlow	16/30 (53.3%)	18.00	1.81
learned task-router	15/30 (50.0%)	17.00	1.71
strong-model-only	15/30 (50.0%)	16.50	1.66
budget-only	12/30 (40.0%)	13.00	1.31
cheap-model-only	11/30 (36.7%)	12.00	1.21

Table 6. Task Value sensitivity on the latest audited run.

Value profile	BF TRV	Control TRV	Margin
equal	16.00	15.00	+1.00
criticality	18.00	17.00	+1.00
compressed	17.00	16.00	+1.00
expanded	20.00	19.00	+1.00

Boundary-aware reporting. The four paid runs show different frontier leaders. This is the intended evidence shape for Batch-Level Task Budget Allocation. A value-aware policy can improve TRV and TRV per Dollar, a strong model can define the best boundary in favorable conditions,

Table 7. KV Cache Cost-Discount sensitivity. Values are TRV per Dollar.

KV profile	BF	Best control	BF margin
KV0	1.81	1.71	+0.10
KV50	3.26	3.14	+0.11
KV90	9.06	8.98	+0.09
KV98	14.08	14.28	-0.20
KV99	15.13	15.41	-0.29

and a learned task-router can be a strong control when value placement is less important. Reporting the frontier makes these cases visible.

Future work. First, BudgetFlow can study finer-grained allocation inside tasks, including stage-aware routing, segment-level routing, and escalation. Second, BudgetFlow can study continual policy learning, where completed runs update future stop/continue and allocation decisions while preserving auditability. Third, BudgetFlow can become serving-aware by connecting Task Value, remaining budget, Estimated Token Demand, and verified outcome history to

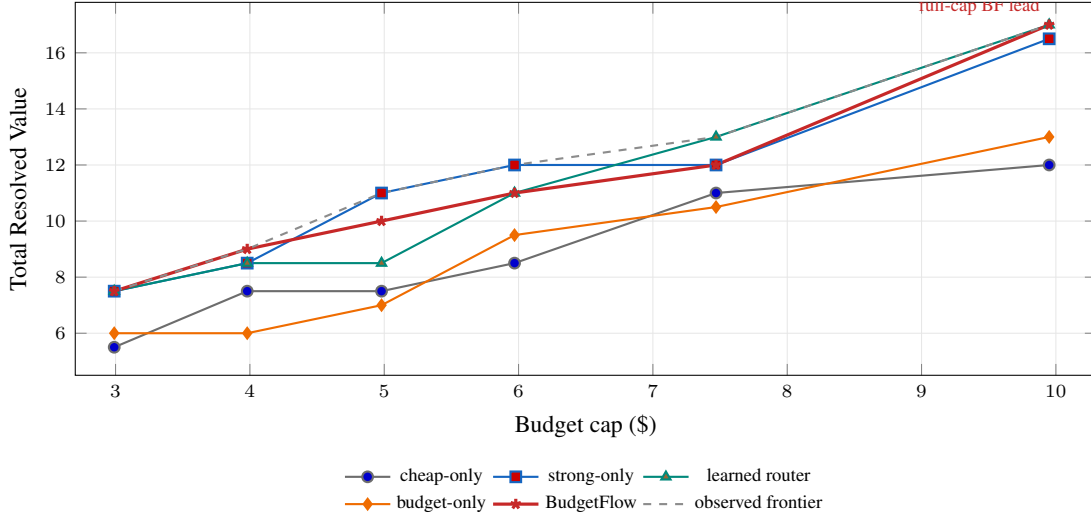


Figure 2. Budget sensitivity diagnostic on the latest audited run. Outcomes are fixed; budget accounting is replayed at alternate caps.

serving substrates such as vLLM, SGLang, and Dynamo. Fourth, BudgetFlow can extend from one budget owner to multi-tenant task budget governance, where multiple teams, users, or services share an agent execution substrate with distinct budgets and priorities.

9. Conclusion

BudgetFlow formulates Batch-Level Task Budget Allocation for multi-step agent tasks under a shared hard budget. The framework uses Task Value, Estimated Token Demand, Model Fit, and a Verifier to connect budget allocation with verified outcomes. On SWE-bench-style coding tasks, BudgetFlow improves all three primary metrics in the latest audited 5-policy run and exposes boundary conditions across multiple paid runs. The central result is a cost-value frontier view of task-batch budgeting: the useful question is how verified value changes under fixed resource constraints and strong controls.

References

- Dekoninck, J., Baader, M., and Vechev, M. A unified approach to routing and cascading for LLMs. In *International Conference on Machine Learning*, 2025. URL <https://arxiv.org/abs/2410.10347>.
- Guo, D., Wu, J., and Yiu, S. M. RouteNLP: Closed-loop LLM routing with conformal cascading and distillation co-optimization. In *ACL Industry Track*, 2026. URL <https://arxiv.org/abs/2604.23577>.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Kotte, V. UCCI: Calibrated uncertainty for cost-optimal LLM cascade routing. *arXiv preprint arXiv:2605.18796*, 2026. URL <https://arxiv.org/abs/2605.18796>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *ACM Symposium on Operating Systems Principles*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Liu, H., Tian, C., An, N., Wang, Z., Lu, P., Yu, C., and Qi, Q. Budget-constrained agentic large language models: Intention-based planning for costly tool use. *arXiv preprint arXiv:2602.11541*, 2026. URL <https://arxiv.org/abs/2602.11541>.
- NVIDIA. NVIDIA Dynamo: Agentic inference. <https://docs.nvidia.com/dynamo/digest/agentic-inference>, 2026. Accessed 2026-07-06.
- Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. RouteLLM: Learning to route LLMs with preference data. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2406.18665>.
- OpenSquilla contributors. OpenSquilla: Token-efficient ai agent. <https://github.com/opensquilla/opensquilla>, 2026. Accessed 2026-07-06.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2312.07104>.

Zheng, M., Han, K., Li, B., Xu, H., Tian, Y., He, W., Zhou, H., Guo, J., Hu, H., Ma, L., Xu, C., Dai, G., Xia, L., Wei, Y., Wang, Y., and Wang, Y. Claw-SWE-Bench: A benchmark for evaluating openclaw-style agent harnesses on coding tasks. *arXiv preprint arXiv:2606.12344*, 2026. URL <https://arxiv.org/abs/2606.12344>.